Санкт-Петербургский государственный университет

Искусственный интеллект и наука о данных

Группа 23.Б16-мм

Реализация и сравнение алгоритмов проверки изоморфизма гиперграфов отношений на Python и C++

Сидоров Вячеслав Павлович

Отчёт по учебной практике
в форме «Производственное задание»

Научный руководитель:
д. ф.-м. н., профессор Косовская Т. М.

Санкт-Петербург
2026

Оглавление

Введение **3**

1. Постановка задачи **4**

2. Обзор **5**

2.1. Гиперграфы отношений 5

2.2. Характеристики и структурное отображение 5

2.3. Алгоритмы ISOM-1 и ISOM-n 6

2.4. Пример работы ISOM-n 8

2.5. Существующие подходы и выбор средств реализации . . 8

2.6. Выводы 9

3. Реализация **10**

3.1. Реализация на Python 10

3.2. Реализация на C++ 12

3.3. Средства сравнения 13

4. Эксперимент **15**

4.1. Исследовательские вопросы и метрики 15

4.2. Условия эксперимента 15

4.3. Наборы задач 16

4.4. Результаты 17

4.5. Обсуждение результатов 19

4.6. Воспроизводимость эксперимента 20

Заключение **21**

Список литературы **22**

Введение

Сложные структурированные объекты — сцены, молекулы, схемы — удобно описывать как множества элементов, обладающих некоторыми свойствами и связанных некоторыми отношениями. Традиционный язык таких описаний — формулы исчисления предикатов. Задачи распознавания в такой постановке NP-трудны [4], поэтому Т. М. Косовская с соавторами предложили выделять общие для класса объектов подформулы-шаблоны; в основе подхода лежит понятие изоморфизма элементарных конъюнкций через шаблон с переменными [11, 6].

У формул исчисления предикатов есть неудобство: каждое отношение имеет фиксированную арность и упорядоченные аргументы, так что утверждение «пятеро людей — братья» требует 20 атомарных формул. Поэтому в работе [3] предложены *гиперграфы отношений*, где коммутативное отношение любой арности задаётся одним гиперребром, а в статье [2] — алгоритмы проверки их изоморфизма ISOM-1 и ISOM-n. Алгоритмы описаны словесно и блок-схемами и проиллюстрированы примерами; программной реализации и экспериментальной оценки у них пока нет.

Чтобы пользоваться алгоритмами, их нужно реализовать, проверить реализацию на независимом эталоне и понять, как время работы зависит от вида входных данных. Важен и выбор языка: на Python алгоритм удобно записывать близко к тексту статьи и исследовать, а на C++ он работает быстрее. Интересно, насколько велика эта разница, если обе реализации выполняют в точности один и тот же поиск.

В работе алгоритмы статьи реализованы на Python и на C++ так, что обе реализации строят одинаковые деревья поиска. Корректность проверена сверкой с полным перебором и сверкой реализаций между собой, а производительность сравнена на десяти наборах задач, от примеров из статьи до гиперграфов с сотнями вершин.

1. Постановка задачи

Целью работы является реализация на языках Python и C++ алгоритмов проверки изоморфизма гиперграфов отношений из статьи Т. М. Косовской [2] и экспериментальное сравнение этих реализаций. Для её выполнения были поставлены следующие задачи:

1. разобрать алгоритмы ISOM-1 и ISOM-n (раздел 2);
2. реализовать алгоритмы на Python (раздел 3.1);
3. реализовать алгоритмы на C++ так, чтобы реализация выполняла тот же поиск, что и реализация на Python (раздел 3.2);
4. разработать наборы тестовых задач и средства автоматического сравнения реализаций (раздел 3.3);
5. проверить корректность реализаций и сравнить их время работы (раздел 4).

2. Обзор

В разделе изложены определения и алгоритмы статьи [2], на которых основана работа, а также известные подходы к проверке изоморфизма графов.

2.1. Гиперграфы отношений

Пусть на множестве $\omega = \{\omega_1, \ldots, \omega_t\}$ заданы свойства и отношения $p_1, \ldots, p_n$. Гиперграф отношений [3] имеет *предикатные* вершины, помеченные именами отношений, и *объектные*, помеченные элементами ω. Ребро типа (1) $p^k \to (\omega_{i_1}, \ldots, \omega_{i_k})$ соединяет вершину некоммутативного k-местного отношения с упорядоченным набором различных объектных вершин, ребро типа (2) $p^{c,k} \to \{\omega_{i_1}, \ldots, \omega_{i_k}\}$ — вершину коммутативного отношения с их множеством. Так, пятеро братьев описываются одним ребром $brothers^{c,5} \to \{\omega_1, \ldots, \omega_5\}$.

Определение 1 ([2])**.** Гиперграфы отношений G_1 и G_2 изоморфны, если существуют гиперграф H, у которого вместо имён объектных вершин стоят переменные, и подстановки (*унификаторы*) λ_1, λ_2 имён объектных вершин вместо переменных такие, что $H\lambda_1 = G_1$ и $H\lambda_2 = G_2$.

На практике H получают из G_2, заменяя различные константы $b_1, \ldots, b_n$ различными переменными $x_1, \ldots, x_n$, и ищут биекцию λ переменных H на объектные вершины G_1, для которой $H\lambda = G_1$. Задача проверки изоморфизма таких гиперграфов полиномиально эквивалентна задаче изоморфизма графов [2, 5].

2.2. Характеристики и структурное отображение

Пусть гиперграф содержит одну предикатную вершину. *Характеристика* вершины a для рёбер типа (1) — вектор $\chi(a) = (n_1, \ldots, n_k)$, где n_j — число вхождений a на j-е место; для рёбер типа (2) — число вхождений a в рёбра. *Характеристика гиперграфа* $\chi(G)$ — упорядочен-

ный набор характеристик его вершин; для проверки равенства годится любой канонический порядок, например лексикографический.

Теорема 1 ([2])**.** *Для изоморфизма гиперграфов с одной предикатной вершиной необходимо, чтобы их характеристики совпадали.*

Вершины с одинаковыми характеристиками образуют классы. *Структурное отображение* $\{[x_{i_1}, \ldots, x_{i_{t_1}}] : [a_{j_1}, \ldots, a_{j_{t_1}}], \ldots\}$ сопоставляет каждому классу переменных H класс констант G с той же характеристикой. Переменные класса могут переходить только в константы того же класса, поэтому вместо $n!$ подстановок достаточно перебрать $\prod_{k=1}^{K} t_k!$, где $t_1, \ldots, t_K$ — размеры классов. Например, в первом примере статьи (отношение p^3, шесть вершин) отображение имеет вид $\{[x_5] : [a_4],\ [x_3] : [a_3],\ [x_2, x_6] : [a_1, a_5],\ [x_1, x_4] : [a_2, a_6]\}$, и вместо $6! = 720$ подстановок остаётся 4.

2.3. Алгоритмы ISOM-1 и ISOM-n

Алгоритм ISOM-1 проверяет изоморфизм гиперграфов G_1 и H с одной предикатной вершиной. Он сравнивает характеристики (шаги 1–2), строит структурное отображение (шаг 3) и перебирает совместимые с ним подстановки λ, проверяя $H\lambda = G_1$ (шаги 4–7). Подстановку отбрасывают досрочно, если часть $H\lambda$, содержащая уже известную константу, не согласуется с соответствующей частью G_1 (шаги 8–12). При известном отображении сложность составляет $O\bigl(\prod_{k=1}^{K} t_k!\bigr)$. Алгоритм без шагов 1–3, получающий готовое отображение, называется ISOM^{-}-1.

Пусть теперь предикатных вершин несколько, и G^i, H^i — подграфы, содержащие только i-ю из них. Сначала сравниваются характеристики всех пар G^i, H^i. Затем алгоритм ISOM-n строит дерево частичных унификаторов: тройки $\langle G, H, \lambda\rangle$ хранятся в очереди, упорядоченной по сложности перебора, и обрабатываются так (рис. 1):

1. MAP: построить отображения $mapping^i$ для подграфов, в которых остались переменные;
2. BESTMAP: выбрать подграф i_0 с наименьшей сложностью $\prod t_k!$;

3. ISOM^{-}-1: найти локальные унификаторы $\hat{\lambda}_1^{i_0}, \ldots, \hat{\lambda}_r^{i_0}$ подграфов G^{i_0} и $H^{i_0}\lambda$;

4. CONSISTENCY: оставить те из них, которые вместе с λ образуют биекцию (нет пар $x/a,\ x/b$ и $x/a,\ y/a$ при $x \neq y$, $a \neq b$);

5. REDUCTION: для каждого оставшегося λ_t сравнить с G^j все подграфы $H^j\lambda\lambda_t$ без переменных; при несовпадении λ_t отбрасывается, а если переменных не осталось нигде, $\lambda\lambda_t$ — унификатор, иначе тройка $\langle G, H, \lambda\lambda_t\rangle$ ставится в очередь.

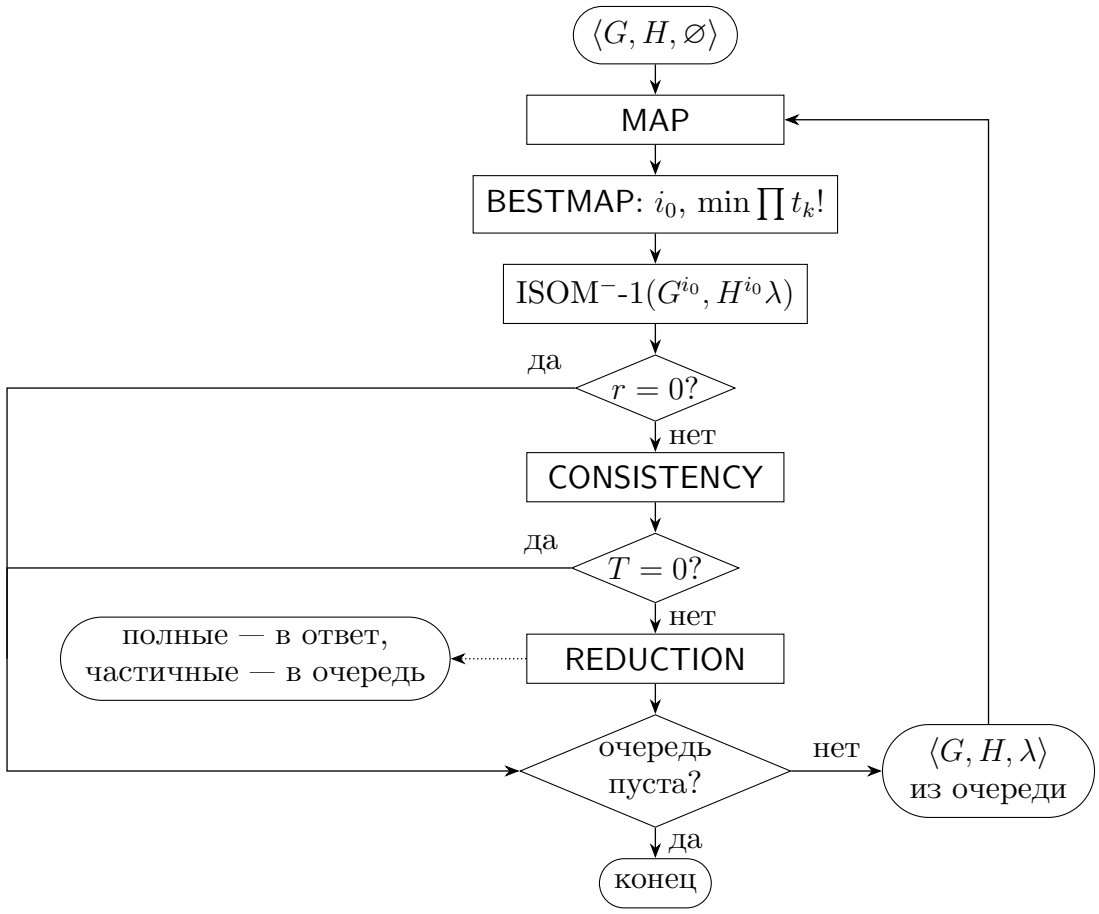


Рис. 1: Блок-схема алгоритма ISOM-n (по рис. 3 из [2])

2.4. Пример работы ISOM-n

Рассмотрим гиперграфы

$$G_1 = \{p_1 \to a_1,\ p_1 \to a_2,\ p_1 \to a_3,\ p_2^2 \to (a_1, a_2),\ p_2^2 \to (a_1, a_4),\ p_3^{c,3} \to \{a_2, a_3, a_4\}\},$$
$$G_2 = \{p_1 \to b_2,\ p_1 \to b_3,\ p_1 \to b_4,\ p_2^2 \to (b_4, b_3),\ p_2^2 \to (b_4, b_1),\ p_3^{c,3} \to \{b_1, b_2, b_3\}\}$$

и шаблон H, полученный из G_2 заменой b_i на x_i. BESTMAP выбирает подграф p_2^2 со сложностью $1! \cdot 2! = 2$; из двух его локальных унификаторов один отсекается на шаге MAP, второй после обработки p_1 даёт унификатор (рис. 2).

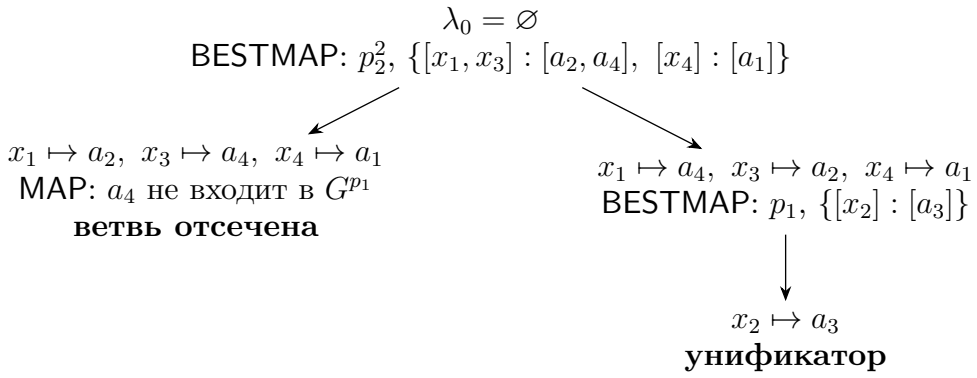


Рис. 2: Дерево поиска ISOM-n для примера из раздела 2.4

2.5. Существующие подходы и выбор средств реализации

Задача изоморфизма графов не доказана ни полиномиальной, ни NP-полной; лучший известный алгоритм квазиполиномиален [1]. На практике используют программы канонической разметки, такие как nauty и Traces [8], основанные на итеративном уточнении разбиения вершин, и перебор с отсечениями, например VF2 [10]. Изоморфизм гиперграфов сводится к изоморфизму двудольного графа инцидентности [7]; для рёбер типа (1) понадобились бы дополнительные вершины для

номеров мест. Подход статьи [2] работает с гиперграфами отношений непосредственно: характеристики делят вершины на классы, перебор идёт по перестановкам внутри классов, а порядок обработки подграфов выбирается по сложности перебора.

Python позволяет записать алгоритм почти дословно по статье: подстановка — словарь, мультимножество рёбер — `Counter`, перебор перестановок — `itertools.permutations`; но интерпретатор медленный. C++17 позволяет хранить данные в компактных массивах, а нужные средства (`std::next_permutation`, кучи) есть в стандартной библиотеке. Чтобы сравнение было честным, от C++-версии требуется обходить дерево поиска и перебирать перестановки в том же порядке, что и Python-версия.

2.6. Выводы

Алгоритм ISOM-n сочетает необходимое условие изоморфизма, сокращение перебора за счёт классов вершин и поиск по дереву частичных унификаторов с выбором самого дешёвого подграфа. Сложность перебора определяется размерами классов, поэтому время работы должно сильно зависеть от структуры входа; это нужно учесть при выборе тестовых задач. Для проверки реализаций нужен независимый эталон; в работе им служит полный перебор всех $n!$ биекций.

3. Реализация

Программная часть состоит из реализации на Python (`hypergraph_isomorphism.py`, 634 строки), реализации на C++ (`hypergraph_isomorphism.cpp`, 993 строки), сценария сравнения `compare.py` (561 строка) и файла `examples.txt` с примерами статьи. Обе реализации читают задачи в одном текстовом формате: строки `pred p2 2 n` объявляют предикаты (арность, тип рёбер), строки `g1 p2 a1 a2` и `g2 ...` — рёбра G_1 и G_2. Шаблон H получается из G_2 заменой вершин, упорядоченных «естественно» ($b_2 < b_{10}$), на $x_1, x_2, \ldots$; ищутся все унификаторы λ с $H\lambda = G_1$, и каждый печатается как изоморфизм $G_2 \to G_1$. Части программы названы так же, как в статье: характеристики, структурное отображение, ISOM^{-}-1, ISOM-1, MAP, BESTMAP, CONSISTENCY, REDUCTION и цикл ISOM-n; для проверки добавлен полный перебор.

3.1. Реализация на Python

Данные. Предикат — неизменяемый класс `Pred` (имя, арность, коммутативность), ребро `Edge` — предикат и кортеж (тип (1)) или `frozenset` (тип (2)) аргументов, гиперграф — `Counter` рёбер. Переменные шаблона — объекты класса `Var`, подстановка — словарь «переменная $\to$ константа».

Структурное отображение. Функция `build_mapping` строит отображение между G^i и $H^i\lambda$, где часть переменных уже заменена константами, только если выполнены необходимые условия: число рёбер и мультимножества характеристик совпадают, а каждая константа, уже стоящая в $H^i\lambda$, имеет в G^i ту же характеристику. Иначе ветвь поиска отсекается.

ISOM^{-}-1. Подстановки перебираются по классам: для класса k перебираются перестановки его констант, и для каждой рекурсивно обрабатываются следующие классы (листинг 1). Ветвь отсекается, как только ребро $H^i\lambda$ без переменных не встречается в G^i (аналог шагов 8–12

ISOM-1), причём каждое ребро проверяется один раз — после назначения того класса, на котором в нём не остаётся переменных. На листе дополнительно проверяется $H^i\lambda = G^i$ (шаг 7).

Лемма 1. *Проверка по уровням отсекает те же подстановки, что и проверка всех рёбер без переменных после каждого класса.*

Доказательство. Ребро, ставшее рёбром без переменных на классе $k' < k$, уже проверено с теми же значениями переменных. Константы, уже стоящие в $H^i\lambda$, исключены из классов, а внутри класса значения образуют перестановку, поэтому подстановка инъективна на вершинах $H^i\lambda$ и разные рёбра переходят в разные; значит, кратности рёбер можно сравнивать поштучно. □

Листинг 1: Ядро ISOM⁻-1 на Python

```
level = {x: k + 1 for k, (xs, _) in enumerate(classes) for x in xs}
checks = [[] for _ in range(len(classes) + 1)]
for e, m in H_i.items():   # уровень ребра: класс, где кончаются
   переменные
    k = max((level[v] for v in e.args if is_var(v)), default=0)
    checks[k].append((e.args, e.pred.commutative, m))

def rec(k):
    if k == len(classes):
        if substitute_edges(H_i, s) == G_i:      # шаг 7
            result.append(dict(s))
        return
    xs, cs = classes[k]
    for perm in itertools.permutations(cs):
        s.update(zip(xs, perm))
        if fits(checks[k + 1]):
            rec(k + 1)
    for x in xs:
        del s[x]
```

ISOM-n. Очередь — двоичная куча `heapq` с ключом «стоимость, номер добавления». MAP и BESTMAP вызываются при добавлении вершины в

очередь, так как её приоритет — стоимость выбранного отображения; при равной стоимости выбирается предикат с меньшим именем. REDUCTION подставляет унификатор только в подграфы, все переменные которых уже заданы. С параметром `verbose=True` печатается журнал поиска, по которому можно восстановить дерево (как на рис. 2).

Лемма 2. *Если BESTMAP выбирает подграф однозначно по λ, то все вершины дерева ISOM-n имеют различные частичные унификаторы, а все найденные унификаторы различны.*

Доказательство. Дети вершины с унификатором λ имеют вид $\lambda\lambda_t$, где все λ_t определены на одном множестве V переменных $H^{i_0}\lambda$ и попарно различны. У потомков унификатор только расширяется, поэтому вершины из поддеревьев разных детей различаются на V, а вершина и её потомок — областью определения. Полные унификаторы получаются из пар «вершина, кандидат» и в очередь не попадают, поэтому они тоже различны. □

Поэтому реализация не хранит множество пройденных состояний, а отсутствие повторов среди унификаторов проверяется при сравнении. Самопроверка модуля сравнивает ISOM-n и ISOM-1 с полным перебором на 400 случайных парах гиперграфов.

3.2. Реализация на C++

Реализация на C++ повторяет Python-версию по шагам, но использует компактные структуры данных. Константы и переменные нумеруются целыми числами в естественном порядке имён, подстановка — массив, где -1 означает «не задано». Рёбра хранятся как строки длины k (у коммутативных строка отсортирована), рёбра G^i — в хеш-таблице с открытой адресацией, характеристики — строки длины k или 1, сравниваемые лексикографически, как кортежи в Python. Стоимость $\prod t_k!$ считается точно в 128-битном целом и при переполнении насыщается. Гиперграф считается множеством рёбер, как в статье. Чтобы деревья

поиска совпадали, совпадают все порядки, от которых зависит обход (табл. 1).

Таблица 1: Порядки, от которых зависит обход, и их реализация

Что упорядочено	Python	C++
перестановки класса	`itertools.permutations` от отсортированного списка	`std::next_permutation` от возрастающего массива
очередь	`heapq`, ключ (стоимость, счётчик)	`std::push_heap`, тот же ключ
предикаты в BESTMAP и REDUCTION	по строке вида `p2^2`	по той же строке (байты UTF-8 упорядочены как коды символов)
классы отображения	по характеристике, затем по имени	по строке характеристики, затем по номеру

Программа печатает для каждой задачи ответ, время (все замеры серии, их среднее и стандартное отклонение), статистику поиска и найденные изоморфизмы; опция `-v` печатает журнал в том же формате, что и Python-версия, `--algo` выбирает ISOM-n, ISOM-1 или полный перебор. Сборка: `g++ -O2 -std=c++17`.

3.3. Средства сравнения

Сценарий `compare.py` генерирует наборы задач (раздел 4.3), запускает обе реализации и для каждой задачи проверяет, что:

- совпадают ответ и множества найденных изоморфизмов, а повторов среди унификаторов нет;
- совпадает статистика дерева поиска: вызовы MAP и отсечения на нём, число вершин (вызовов ISOM$^-$-1), суммы r и T, число полных и частичных унификаторов; в Python она собирается обёртками, временно подставленными вместо функций модуля;
- на малых задачах полный перебор обеих реализаций и ISOM-1 (если предикат один) дают то же множество, что и ISOM-n;
- на примерах статьи журналы поиска совпадают построчно.

Время замеряется для поиска всех унификаторов ISOM-n без чтения входа, по методике из раздела 4.2; сырые замеры сохраняются для анализа.

4. Эксперимент

4.1. Исследовательские вопросы и метрики

RQ1: верны ли результаты реализаций и выполняют ли они один и тот же поиск?

RQ2: во сколько раз реализация на C++ быстрее реализации на Python и зависит ли это от вида задачи?

RQ3: как время работы ISOM-n зависит от размера и структуры входа?

Корректность оценивается совпадением ответов, множеств изоморфизмов и статистики дерева поиска у двух реализаций, а на малых задачах — ещё и с полным перебором. Производительность оценивается временем работы и ускорением $t_{\text{Python}}/t_{\text{C++}}$.

4.2. Условия эксперимента

Замеры выполнены на ноутбуке с процессором Intel Core i5-10210U (4 ядра, 8 потоков) и 14 ГБ памяти под Ubuntu 26.04.1 LTS; использовались CPython 3.14.4 и g++ 15.2.0 (`-O2 -std=c++17`). На время замеров частота процессора зафиксирована на базовом значении 1,6 ГГц, турбобуст отключён, процесс закреплён за одним ядром (`taskset`). Обе реализации однопоточные, сборщик мусора Python не отключался.

Замеры проводились по методике [12]. Для каждой задачи после прогревочного запуска выполняется серия из 40 замеров. Задачи короче 20 мс, как в модуле `timeit` [9], замеряются пачками запусков, и замером считается среднее время одного запуска в пачке; в наборах `random` и `random-mid` из сотен мелких задач замером служит время решения всех задач набора подряд. По серии вычисляются среднее и стандартное отклонение (с.о.); по методике с.о. должно быть меньше 10 % среднего, а одиночный выброс можно отбросить. Серия, не удовлетворяющая этому условию, замерялась заново. Из 112 серий отброшено 19 одиночных выбросов и повторно замерено 27 серий; отношение с.о. к среднему в

итоге составило в медиане 6,5 % для Python и 3,4 % для C++, в худшем случае — 9,9 % и 9,8 %. Гистограммы замеров одновершинные, с обычным для замеров времени хвостом в сторону больших значений. Реализации различаются по времени примерно на два порядка, поэтому проверка нормальности и доверительные интервалы, которые методика требует для программ с близким временем работы, не нужны; погрешность ускорения не превышает 14 %.

4.3. Наборы задач

Задачи генерирует `compare.py` с фиксированным зерном (табл. 2). Наборы покрывают разные режимы работы алгоритма: поиск без ветвлений, факториальный перебор внутри класса, множество унификаторов и большое дерево частичных унификаторов. У структурированных наборов есть неизоморфные варианты с совпадающими характеристиками, которые нельзя отбросить предварительной проверкой.

Таблица 2: Наборы задач. n — число вершин, $|E|$ — число рёбер G_1

Набор	Содержание	Размер	Изоморфны
`paper`	примеры из статьи и их варианты, включая «пять братьев»	$n = 4$–6	4 из 5
`random`	300 случайных пар с 1–3 предикатами	$n \le 7$	188 из 300
`random-mid`	100 случайных пар с 3–5 предикатами	$n = 8$–20	74 из 100
`rigid`	отношение r^3, все характеристики вершин которого различны, и рёбра $s^{c,2}$, $t^{c,3}$, p^1	$n = 24$–768, $\|E\| \le 4860$	все
`rigid-non`	как `rigid`, но у двух рёбер r^3 в G_2 обменяны аргументы на одном месте	то же	нет
`cycle`	ориентированный цикл $q^2(v_i, v_{i+1})$, все характеристики равны $(1, 1)$	$n = 4$–9	все, n унификаторов
`cycle-non`	цикл длины n против двух циклов	$n = 4$–9	нет
`brothers`	одно ребро $brothers^{c,k}$	$k = 3$–8	все, $k!$ унификаторов
`ladder`	«лестница»: два цикла длины d и d перекладин $p_j^{c,2}$ со своими предикатами	$d = 2$–11, $n = 2d$	все, 2 унификатора
`ladder-non`	второй цикл обходит вершины в другом порядке	$d = 3$–11	нет

В наборе `rigid` характеристики r^3 выбираются вместе с циклическими сдвигами, поэтому суммы по местам совпадают и все классы отображения одноэлементны. В наборе `ladder` перестановку концов перекладины не различает ни одна характеристика, поэтому обе подста-

новки каждой перекладины проходят CONSISTENCY и REDUCTION, и дерево поиска содержит $2^d - 1$ вершин.

4.4. Результаты

4.4.1. RQ1: корректность

Сценарий сравнения выполнил 3095 проверок на 454 задачах; число расхождений — 0. Совпали ответы, множества изоморфизмов и статистика деревьев поиска на всех задачах, журналы на примерах статьи, а на наборах `paper` и `random` — результаты полного перебора и ISOM-1. Кроме того, самопроверка Python-модуля и ещё 1500 случайных пар с $n \leq 8$ совпали с полным перебором, режим поиска до первого унификатора совпал в обеих реализациях на 307 задачах, а C++-версия, собранная с флагом `-fsanitize=address,undefined`, отработала на всех наборах без ошибок. Совпадение статистики означает, что реализации обходят одни и те же вершины дерева и рассматривают одних и тех же кандидатов.

4.4.2. RQ2: ускорение

Времена на самой большой задаче каждого набора приведены в табл. 3, ускорение по всем задачам — на рис. 3. Оно лежит в пределах от 56 до 144 раз, среднее геометрическое — 84 раз. Меньше всего ускорение на мелких случайных задачах (63 и 60 раз на наборах целиком), где в Python заметную долю времени занимают вызовы функций и создание объектов. Больше всего — на наборе `brothers` (в среднем 123 раз), где почти всё время уходит на перебор перестановок и копирование подстановок.

4.4.3. RQ3: зависимость от размера и структуры

Рис. 4 показывает рост времени с размером задачи. Кривые Python и C++ в логарифмическом масштабе почти параллельны: реализации различаются постоянным множителем, а характер роста задаёт алгоритм.

Таблица 3: Время ISOM-n (мс): среднее ± стандартное отклонение по 40 замерам; «вершин» — число вершин дерева поиска. Для `paper` — сумма по задачам, для `random` и `random-mid` — время решения всех задач набора подряд

Задача	n	$\lvert E\rvert$	Вершин	Python	C++	Ускорение
`rigid_n768`	768	4860	1	203 ± 8	2,59 ± 0,02	78×
`rigid_non_n768`	768	4860	1	151 ± 7	2,202 ± 0,013	69×
`cycle_n9`	9	9	1	1856 ± 12	21,0 ± 0,9	88×
`cycle_non_n9`	9	9	1	1840 ± 15	21,3 ± 0,3	86×
`brothers_k8`	8	1	1	3110 ± 50	25,4 ± 0,2	122×
`ladder_d11`	22	33	2047	1447 ± 15	15,73 ± 0,18	92×
`ladder_non_d11`	22	33	2047	1453 ± 20	15,7 ± 0,4	92×
`paper`: 5 задач	4–6			9,63 ± 0,12	0,082 ± 0,003	118×
`random`: 300 задач	1–7			158 ± 12	2,52 ± 0,12	63×
`random-mid`: 100 задач	8–20			430 ± 20	7,2 ± 0,2	60×

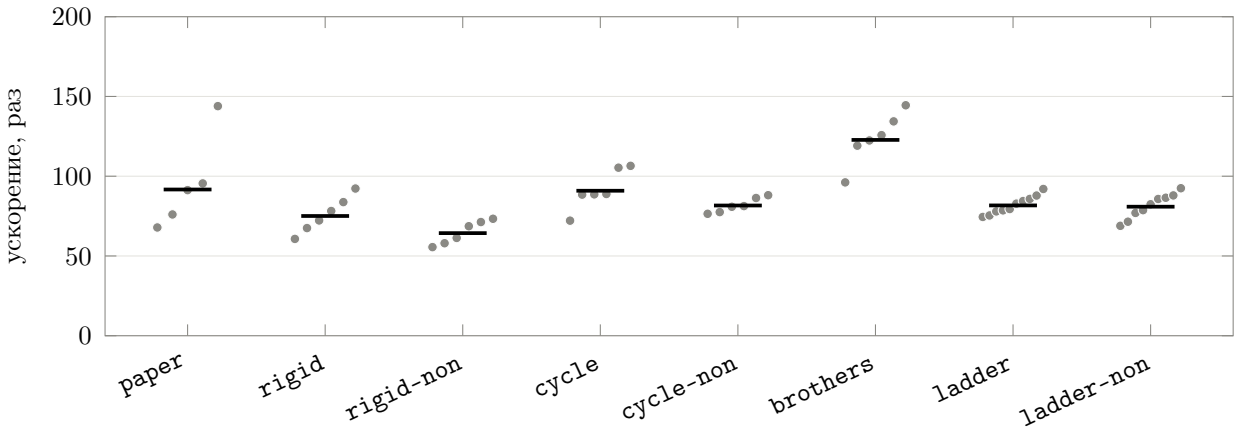


Рис. 3: Ускорение C++ относительно Python на каждой задаче (точки) и среднее геометрическое по набору (черта)

- `rigid`: классы одноэлементны, поиск идёт без ветвлений; при удвоении n с 384 до 768 число рёбер растёт в 2,5 раза, время — в 2,4 (Python) и 2,2 (C++) раза, то есть почти линейно по размеру входа.
- `cycle`: все вершины в одном классе, ISOM$^-$-1 перебирает $n!$ перестановок; от $n = 8$ к $n = 9$ время растёт в 9,5 и 9,5 раза, в согласии с оценкой $O(\prod t_k!)$.
- `brothers`: все $k!$ перестановок — унификаторы; от $k = 7$ к $k = 8$ время растёт в 8,9 и 8,7 раза.
- `ladder`: дерево удваивается с каждой перекладиной, время растёт

в 2,1 и 2,1 раза от $d = 10$ к $d = 11$, то есть линейно по числу вершин дерева.

Неизоморфные варианты ведут себя так же, как изоморфные, так как поиск доходит до конца.

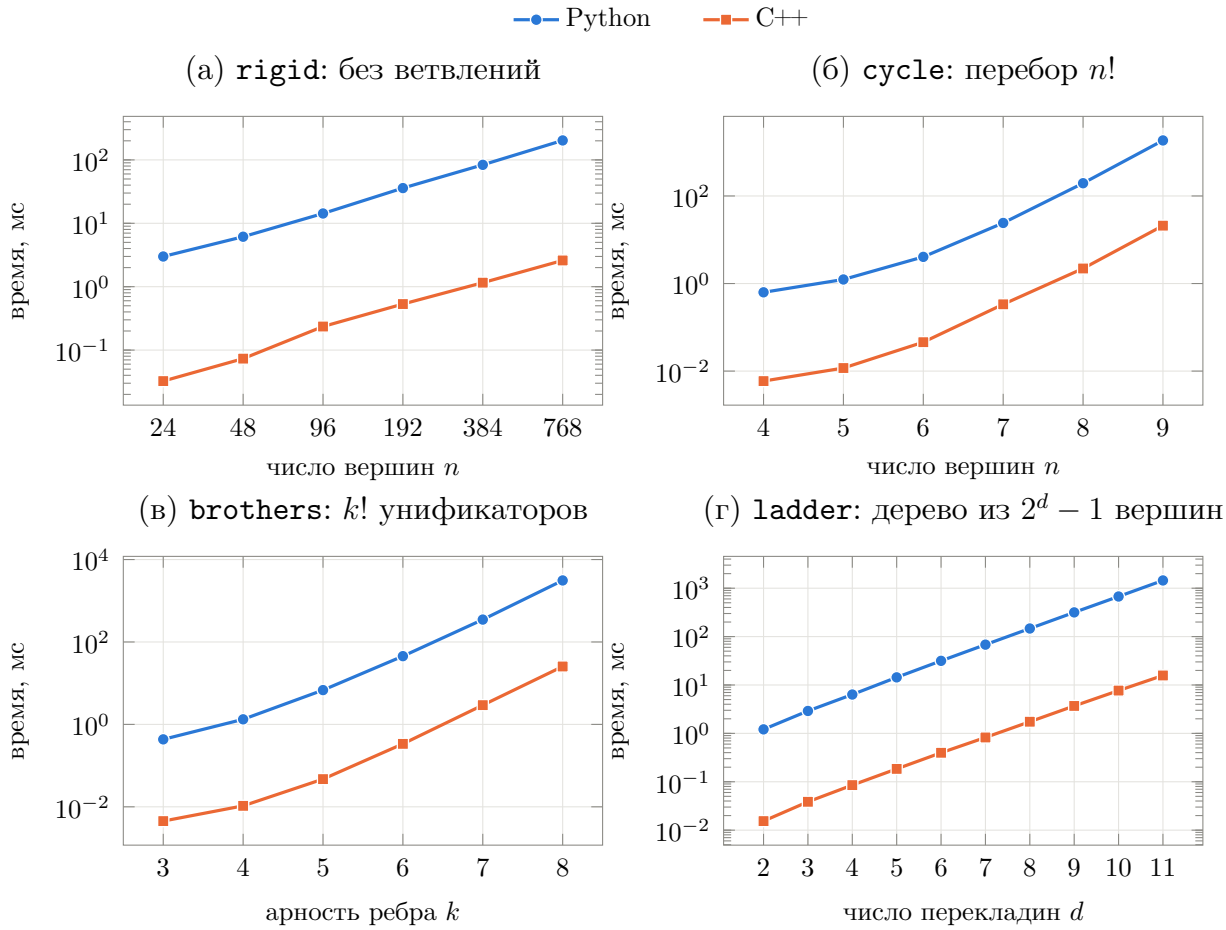


Рис. 4: Время ISOM-n от размера задачи (логарифмическая шкала времени, на графике (а) — и по n). Python — синие круги, C++ — оранжевые квадраты

4.5. Обсуждение результатов

При одинаковом поиске реализация на C++ быстрее в среднем в 84 раз, и эта разница почти постоянна: в Python основная доля времени уходит на элементарные операции интерпретатора (создание кортежей и словарей, поиск в хеш-таблицах, вызовы функций), а в C++ те же операции выполняются над целыми числами в непрерывных массивах.

Время работы ISOM-n, как и следует из оценки $O(\prod t_k!)$, определяется размерами классов вершин: если характеристики различают вершины, оно почти линейно по размеру входа, а на регулярных структурах растёт факториально или экспоненциально. Направления развития — итеративное уточнение классов по характеристикам соседей, как в программах канонической разметки [8], и учёт уже занятых констант при построении отображения; описанные реализации могут служить эталоном для их проверки.

4.6. Воспроизводимость эксперимента

Замеры воспроизводятся командой `python3 compare.py` из корня проекта (около 13 минут при частоте 1,6 ГГц): она собирает C++-программу, генерирует задачи с зерном 2026 и пишет результаты в каталог `bench`.

Заключение

В ходе работы получены следующие результаты.

- Разобраны алгоритмы ISOM-1 и ISOM-n из статьи [2]; показано, что при однозначном выборе подграфа в BESTMAP все вершины дерева поиска различны.
- Алгоритмы реализованы на Python вместе с пошаговым журналом поиска и самопроверкой по полному перебору.
- Алгоритмы реализованы на C++17 так, что реализация обходит дерево поиска в том же порядке и выдаёт тот же журнал, что и Python-версия.
- Разработан сценарий сравнения с десятью наборами задач, проверяющий ответы, множества изоморфизмов, статистику деревьев поиска, журналы и согласие с полным перебором.
- Реализации проверены на 454 задачах (3095 проверок, расхождений нет). Реализация на C++ быстрее в 56–144 раз, в среднем в 84 раз, и ускорение мало зависит от размера задачи.

Исходный код находится в каталоге проекта рядом с отчётом: реализации `hypergraph_isomorphism.py` и `hypergraph_isomorphism.cpp`, сценарий сравнения `compare.py` и примеры `examples.txt`.

Список литературы

[1] Babai László. Graph Isomorphism in Quasipolynomial Time // Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC 2016). — ACM, 2016. — P. 684–697.

[2] Kosovskaya Tatiana. On the isomorphism of hypergraphs of relations. — Extended version, manuscript.

[3] Kosovskaya Tatiana. Hypergraphs for Describing Complex Structured Objects // Computer Science and Information Technologies: Selected Papers of CSIT-2025, 21–25 September, Yerevan, Armenia. — AIP Conference Proceedings. — 2025.

[4] Kosovskaya T. M. Proofs of Estimates of the Number of Steps for Solving Some Image Recognition Problems with Logical Descriptions // Vestnik of St. Petersburg University. Applied Mathematics. Computer Science. Control Processes. — 2007. — no. 4. — P. 82–90. — (In Russian).

[5] Kosovskaya T. M., Kosovskii N. N. Polynomial Equivalence of the Problems Predicate Formulas Isomorphism and Graph Isomorphism // Vestnik St. Petersburg University, Mathematics. — 2019. — Vol. 52, no. 3. — P. 286–292.

[6] Kosovskaya T. M., Zhou J. Algorithm for Extraction Common Properties of Objects Described in the Predicate Calculus Language with Several Predicate Symbols // Programming and Computer Software. — 2024. — Vol. 50, no. Suppl. 1. — P. S1–S9.

[7] Luks Eugene M. Hypergraph Isomorphism and Structural Equivalence of Boolean Functions // Proceedings of the 31st Annual ACM Symposium on Theory of Computing (STOC 1999). — ACM, 1999. — P. 652–658.

[8] McKay Brendan D., Piperno Adolfo. Practical Graph Isomorphism, II // Journal of Symbolic Computation. — 2014. — Vol. 60. — P. 94–112.

[9] Python Software Foundation. timeit — Measure execution time of small code snippets. — URL: https://docs.python.org/3/library/timeit.html (дата обращения: 29 октября 2026 г.).

[10] A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs / Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, Mario Vento // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2004. — Vol. 26, no. 10. — P. 1367–1372.

[11] Zhou J., Kosovskaya T. M. Algorithm for Extraction Common Properties of Objects Described in the Predicate Calculus Language with a Single Predicate Symbol // Vestnik St. Petersburg University, Mathematics. — 2024. — Vol. 57, no. 4. — P. 514–522.

[12] Смирнов К. К. Шпаргалка о проведении экспериментов. — URL: https://github.com/spbu-se/measurements (дата обращения: 29 октября 2026 г.).